StealthChat: Next-Gen Real-Time Architecture & Steganographic Carrier Security

Bridging TCP/IP Transport Layer Foundations with Novel Camouflaged Web Messaging

Computer Networking (CN) Case Study • Core Innovation & Comprehensive System Architecture

THE CORE INNOVATION

1. Project Motivation: The Fatal Flaw in Modern Chat Apps

Why standard End-to-End Encryption (E2EE) fails against network-level surveillance

The Traditional E2EE Dilemma

Apps like WhatsApp, Signal, and Telegram encrypt message text, but their packets loudly scream 'CIPHERTEXT' to network monitors, ISPs, firewalls, and Deep Packet Inspection (DPI) appliances.

Traffic Analysis & Censorship

In corporate or restricted networks, detecting encrypted payloads leads to packet throttling, blocking, or flagged surveillance—even if the attacker cannot crack the mathematical cipher.

Our Novel Breakthrough (StealthChat)

Instead of transmitting obvious ciphertext, our system invents a dual-layer Steganographic Carrier Channel: confidential data is embedded into procedural image pixels, masking the existence of communication.

Security Through Camouflage

To firewalls, packet sniffers (Wireshark), and server admins, the traffic is identical to normal, benign PNG image sharing. Zero metadata or ciphertext alerts are triggered.

NOVELTY ARCHITECTURE

2. Our Novelty: Canvas LSB Image Encryption

How client-side Least Significant Bit (LSB) steganography conceals confidential data

Procedural Carrier Canvas Generation

The client browser dynamically synthesizes a unique HTML5 Canvas gradient artwork on the fly, avoiding hardcoded or repeated image hashes.

Binary Bit-Level Encoding

Secret text is converted to UTF-8 binary bitstreams, terminated by a 16-bit null delimiter (0000000000000000) to eliminate metadata length headers.

Bitwise Pixel Mutation: Pixel & ~1 | Bit

Bits replace only the Least Significant Bit of Red, Green, and Blue bytes. Maximum distortion per color channel is <= 1/255 (0.39%), below the human visual perception threshold.

Client-Side Zero-Knowledge Extraction

The recipient decrypts purely in their local Canvas pixel buffer. The server, intermediaries, and network logs only ever see standard image/png binary data.

COMPETITIVE ANALYSIS

3. Competitive Analysis: StealthChat vs Industry Giants

Evaluating our novel architecture against leading commercial chat applications

Payload Visibility to DPI

WhatsApp/Signal: Obvious raw ciphertext streams. StealthChat: Fully disguised as harmless PNG image frames—undetectable by Deep Packet Inspection.

Deniable Communication

WhatsApp/Signal: None. Metadata logs prove an encrypted exchange occurred. StealthChat: Supported. The sender can claim they were sharing random digital artwork.

Zero-Plugin Browser Execution

WhatsApp/Signal require heavy compiled native cryptography libraries (libsignal). StealthChat operates 100% in pure HTML5 Canvas & WebSockets without browser plugins.

Transport & Delivery Guarantees

Combines the best of WhatsApp (SENT ✓, DELIVERED ✓✓, READ ✓✓ receipts, Heartbeat presence TTL) with our custom steganographic carrier security.

NETWORK LAYER DEFENSE

4. Network Camouflage & DPI Resistance

Demonstrating how Steganography defeats network-level packet sniffers

Deep Packet Inspection (DPI) Vulnerability

Modern enterprise firewalls and network middleboxes inspect transport payloads. High-entropy ciphertext strings are immediately flagged as suspicious or blocked.

MIME-Type & Framing Disguise

StealthChat formats the steganographic carrier into standard Base64 image/png data, adhering to normal HTTP/WebSocket MIME classifications.

Admin Audit Dashboard Interception Proof

Our built-in Admin Security Dashboard captures intercepted database logs: it confirms that snooping admins and proxy observers see only [BASE64 CARRIER IMAGE].

Dual-Layer Defense in Depth

Layer 1: Protocol Security (WebSockets + JWT Channel ACL) + Layer 2: Novel Payload Camouflage (Visual Steganography).

TRANSPORT LAYER

5. Networking Fundamentals: TCP vs UDP Protocol Choice

Why TCP is mandatory for reliable, ordered multi-client chat applications

TCP (Transmission Control Protocol)

Connection-oriented with strict 3-way handshake (SYN -> SYN-ACK -> ACK). Guarantees packets arrive in exact sequential order without bit corruption or duplication.

UDP (User Datagram Protocol)

Connectionless fire-and-forget protocol. High speed but zero delivery guarantees—packets can arrive out of order, corrupted, or not at all.

Chat Semantic Integrity

A chat message requires 100% reliability. Dropping fragments or reordering packets corrupts meaning. Hence, TCP is the required transport layer foundation.

Transport Layer Mapping

Both our Phase 1 Raw Socket Server and Phase 2 WebSocket Architecture run on top of TCP.

TRANSPORT LAYER

6. Sockets, Ports & Concurrency Architecture

How the transport layer routes data and manages concurrent client connections

Socket Endpoints

A network socket is an endpoint abstraction combining an IP address and port number (e.g., 127.0.0.1:8189), allowing the OS TCP stack to bind packets to specific applications.

The Concurrency Bottleneck

A single-threaded server blocks inside accept() or read() loops. If one client has high latency, all other connected clients are blocked from communicating.

Multi-Threaded Server Delegation

The main server thread exclusively accepts incoming handshakes, offloading each socket to a dedicated ClientHandler background worker thread.

Thread-Safe Mutex Synchronization

Broadcast client sets are protected with synchronized mutex locks, preventing ConcurrentModificationException during simultaneous client connects/disconnects.

PHASE 1 PROTOTYPE

7. Foundation Prototype: Raw Java TCP Socket Chat

Architecture and design of our practical Java multi-threaded socket implementation

ChatServer.java (Core Server)

Initializes ServerSocket(8189), manages persistent client streams, and broadcasts messages across synchronized client output writers.

ChatClient.java (Terminal Client)

Demonstrates dual-thread terminal I/O: Main thread reads stdin; background listener thread monitors socket InputStream for broadcasts.

ChatClientGUI.java (Swing Interface)

Abstracts socket networking behind a GUI. Event Dispatch Thread (EDT) updates visuals while background worker threads handle blocking network calls.

Network Boundary

Operates directly on raw TCP streams; clients communicate via raw plaintext byte streams delimited by newline characters.

APPLICATION LAYER

8. Architectural Shift: Why WebSockets? (RFC 6455)

Overcoming the fundamental limitations of HTTP and raw TCP in web browsers

Browser Sandbox Barrier

Web browsers strictly prohibit raw TCP socket access for security reasons; all traffic must traverse standard web ports (80/443).

HTTP/1.1 Polling Overhead

Periodic polling generates 500B-2KB of redundant HTTP headers per request, flooding network pipes and creating server connection spikes.

HTTP Long-Polling & SSE

Unidirectional and prone to connection timeouts; requires closing and reconnecting HTTP request cycles continuously.

The WebSocket Solution (RFC 6455)

Initiates via standard HTTP 101 Upgrade handshake, then converts into a persistent full-duplex TCP stream with tiny 2-10 byte framing overhead.

DISTRIBUTED SYSTEMS

9. Publish-Subscribe (Pub/Sub) Message Fan-Out

Decoupling message routing across distributed clusters

Multi-Server Connection Problem

In scalable deployments, Sender (on Gateway Node 1) and Recipient (on Gateway Node 2) are not connected to the same server memory space.

Redis / Kafka Pub/Sub Broker

WebSocket gateways connect to a central Pub/Sub broker cluster. When a message is sent, Gateway 1 publishes to the destination channel.

1-on-1 vs Channel Routing

1-on-1 messages publish to user:{recipient_id}. Group messages fan out to /topic/{roomId} where all participating nodes relay to connected users.

Horizontal Scaling

WebSocket gateways become stateless relays; new nodes can be added or destroyed dynamically without disrupting routing logic.

SYSTEM DESIGN

10. Stateless API Microservices Architecture

Separating stateful real-time connections from transactional business logic

Stateful Layer (WebSocket Gateway)

Handles persistent TCP sockets, STOMP heartbeat ping/pong frames, and ephemeral delivery ACKs at /ws-chat.

Stateless HTTP/REST Services

All non-real-time operations remain strictly stateless: Auth Service (/api/auth), Group Management (/api/groups), User Profiles, and Interception Audits (/api/admin).

Independent Elastic Scaling

Stateless REST APIs autoscale horizontally behind standard L7 load balancers (Nginx/HAProxy) based on CPU/traffic without dropping active chat sessions.

Zero-Downtime Deployments

Business logic and profile schemas can be updated without cycling long-lived WebSocket connections.

ESSENTIAL FEATURES

11. Message Delivery State Machine (ACK Tracking)

Building user trust through sent, delivered, and read receipt feedback

Pending (🕑)

Message is created on client and placed in local IndexedDB outbox while offline or in-flight.

Sent (✓ Single Gray Tick)

Server receives frame, validates idempotency, persists to database, and emits an immediate ACK to the sender.

Delivered (✓✓ Double Gray Tick)

Recipient's WebSocket client receives the message frame and automatically emits an ACK frame (/app/chat.ackDelivery/{roomId}).

Read (✓✓ Double Blue Tick)

Recipient opens the active conversation viewport; client triggers /app/chat.ackRead/{roomId}, updating chat bubbles in real-time.

ESSENTIAL FEATURES

12. Presence Status System (Heartbeat + TTL Cache)

Accurate online, away, and offline tracking without socket polling overhead

The Ghost Connection Problem

Abrupt client crashes, tunnel loss, or dead batteries often leave TCP connections half-open without a TCP FIN packet.

Heartbeat Keep-Alive

Active clients emit a lightweight heartbeat ping (/app/chat.heartbeat) every 15 seconds containing presence status (ONLINE or AWAY).

Cache Storage with 30s TTL

PresenceService maintains an in-memory/Redis TTL cache. If no heartbeat arrives within 30 seconds, the user's presence key automatically expires.

Scheduled Eviction Sweep

An asynchronous task sweeps expired records every 5s, immediately notifying channel members of status transitions (/topic/presence).

ESSENTIAL FEATURES

13. Multi-Device & Cross-Tab Synchronization

Zero-redundancy local synchronization across browser sessions

BroadcastChannel API

Browser tabs of the same origin communicate directly using new BroadcastChannel('stealthchat_sync_channel').

Cross-Tab Read Sync

When Tab 1 marks a conversation as READ, it broadcasts an ACK_UPDATE event; Tab 2 instantly updates its tick indicators without making extra network calls.

Outgoing Message Mirroring

Messages sent in one window immediately appear in other active tabs without duplicate REST polling.

Cross-Device Sequence Clock

Monotonically increasing sequence IDs allow mobile and desktop companion devices to fetch missing message deltas upon reconnect.

OFFLINE ARCHITECTURE

14. Offline-First Architecture: IndexedDB & Service Worker

Ensuring continuous chat usability through degraded network conditions

Local IndexedDB Datastore (StealthChatDB)

Maintains local object stores: 'messages' (indexed by roomId/timestamp for instant local rendering) and 'outbox' (for pending queued frames).

Service Worker Shell Caching (sw.js)

Pre-caches HTML, scripts, and stylesheets using a Network-First fallback strategy, allowing full app access while offline.

Background Sync API (sync-queued-messages)

Registers an asynchronous sync tag. The service worker notifies the page when connectivity is restored, triggering an automated outbox drain.

Optimistic UI Rendering

Users compose and view queued messages instantly with a pending clock badge, eliminating UI latency.

SECURITY & ACCESS CONTROL

15. Security: Authentication & Channel-Level ACL

Short-lived JWTs and packet-level STOMP channel access control

Stateless JWT Issuance

Upon successful REST login (/api/auth/login), users receive an RFC 7519 HMAC-SHA256 signed token embedding identity, role, and expiration.

STOMP CONNECT Interception

WebSocketSecurityChannelInterceptor extracts the JWT from native Authorization: Bearer headers and establishes an authenticated StompPrincipal.

STOMP SUBSCRIBE Channel Guard

When a client subscribes to /topic/{roomId}, the interceptor verifies group membership in GroupMembershipRepository. Unauthorized subscriptions are dropped!

Constant-Time Verification

Signature verification uses constant-time byte comparisons to eliminate side-channel timing attack vulnerabilities.

DATA INTEGRITY

16. Client-Side Idempotency & Deduplication

Eliminating duplicate messages caused by unstable network retries

The Retry Storm Dilemma

On mobile connections, a message is often delivered successfully, but the server's return ACK is dropped. The client retries, duplicating the message.

Client Message ID Assignment

Clients generate a cryptographically unique UUIDv7 / timestamp string (clientMsgId) before transmitting.

Sliding-Window Idempotency Cache

IdempotencyService maintains an in-memory cache with a 5-minute TTL window alongside a persistent database index check.

Zero Duplication Guarantee

If a duplicate clientMsgId is detected, the server suppresses duplicate database inserts and broadcasts, returning the existing status safely.

PRACTICAL IMPLEMENTATION

17. Live Demonstration: StealthChat Full-Stack App

Enterprise Spring Boot 3 & modern browser client showcase

Technology Stack

Java 17, Spring Boot 3.2.3, Spring WebSocket MessageBroker (SockJS/STOMP), Spring Data JPA, H2 Database, Vanilla HTML5/CSS3.

WhatsApp-Inspired User Experience

Dark-mode aesthetic, dynamic tick indicators, group invite code system, and responsive layout across desktop and mobile.

Steganographic Carrier Demonstration

Toggle Stealth Mode to view how secret text turns into an aesthetic noise canvas and is decrypted cleanly on the recipient end.

Admin Security Audit Panel

Real-time audit log demonstrates that server databases and interceptors only capture carrier images without plaintext leakage.

ENGINEERING LESSONS

18. Real-World Engineering Challenges Encountered

Key obstacles resolved during development and architectural migration

Port Conflicts (HTTP 400 Mismatch)

Port 8080 was occupied by a background HTTP server that rejected raw TCP text with HTTP 400; resolved by dedicating port 8189 for raw sockets and 8080 for web.

Thread Safety & Race Conditions

Concurrent connections caused ConcurrentModificationException on client lists; resolved using synchronized mutex blocks.

Spring Boot Circular Dependency Cycle

WebSocketConfig -> SecurityInterceptor -> PresenceService -> SimpMessagingTemplate formed an unresolvable cycle; resolved via @Lazy decoupling.

Dual-Tab Message Duplication

BroadcastChannel cross-tab sync initially triggered duplicate DOM bubbles; resolved via unique clientMsgId DOM deduplication.

CONCLUSION & SUMMARY

19. Conclusion: Academic & Technical Contributions

Summary of networking contributions and novel innovations

Transport Layer Mastery

Demonstrated hands-on socket programming, port binding, and thread synchronization in Java.

Enterprise Distributed Systems

Successfully implemented WebSockets, Pub/Sub fanout, Stateless REST microservices, and JWT channel authorization.

Reliability Across Hostile Networks

Solved mobile instability using client idempotency, delivery receipt state machines, and heartbeat TTL presence.

Our Core Novelty (Steganographic Carrier)

Pioneered a deniable, DPI-resistant messaging channel by concealing confidential data inside procedural carrier images—advancing state-of-the-art chat privacy.